Homework 4: Lightfields and Focal Stacks

Andrew Krapivin

Oct 30, 2026

1 Lightfield Rendering, Depth from Focus, and Confocal Stereo (1)

1.1 Initials

I read the 6400×11200 lenslet image with OpenCV, convert it from BGR to RGB, and scale it to $[0, 1]$. Pixel (y, x) lies under lenslet $(s, t) = (\lfloor y/16 \rfloor, \lfloor x/16 \rfloor)$, at position $(u, v) = (y \bmod 16, x \bmod 16)$ within it. Reshaping the image to $(400, 16, 700, 16, 3)$ therefore splits the rows into (s, u) and the columns into (t, v), and transposing the axes to the order $(1, 3, 0, 2, 4)$ gives $L(u, v, s, t, c)$, of shape $16 \times 16 \times 400 \times 700 \times 3$ (`load_lightfield` in `src/main.py`).

1.2 Sub-aperture views

The sub-aperture view at (u, v) is the 400×700 image `L[u, v]`. To build the mosaic, `tile_grid` transposes the 16×16 grid of views to $(16, 400, 16, 700, 3)$ and reshapes it to $6400 \times 11200 \times 3$, so view (u, v) is the tile in row u and column v (Fig. 1). This is the inverse of the rearrangement above, applied with the roles of (u, v) and (s, t) swapped.

Neighboring views differ only by a small parallax. Measured with phase correlation against the center view $(8, 8)$, the scene in view $(0, 8)$ is shifted about 8 pixels up and in view $(15, 8)$ about 7 pixels down, so it moves down as u increases. Along v it moves the opposite way: view $(8, 0)$ is shifted about 8 pixels to the right, so the scene moves left as v increases. This mirrored v axis matters for refocusing. The views have nearly the same brightness (mean intensity between 0.35 and 0.38), so the lightfield has little vignetting.

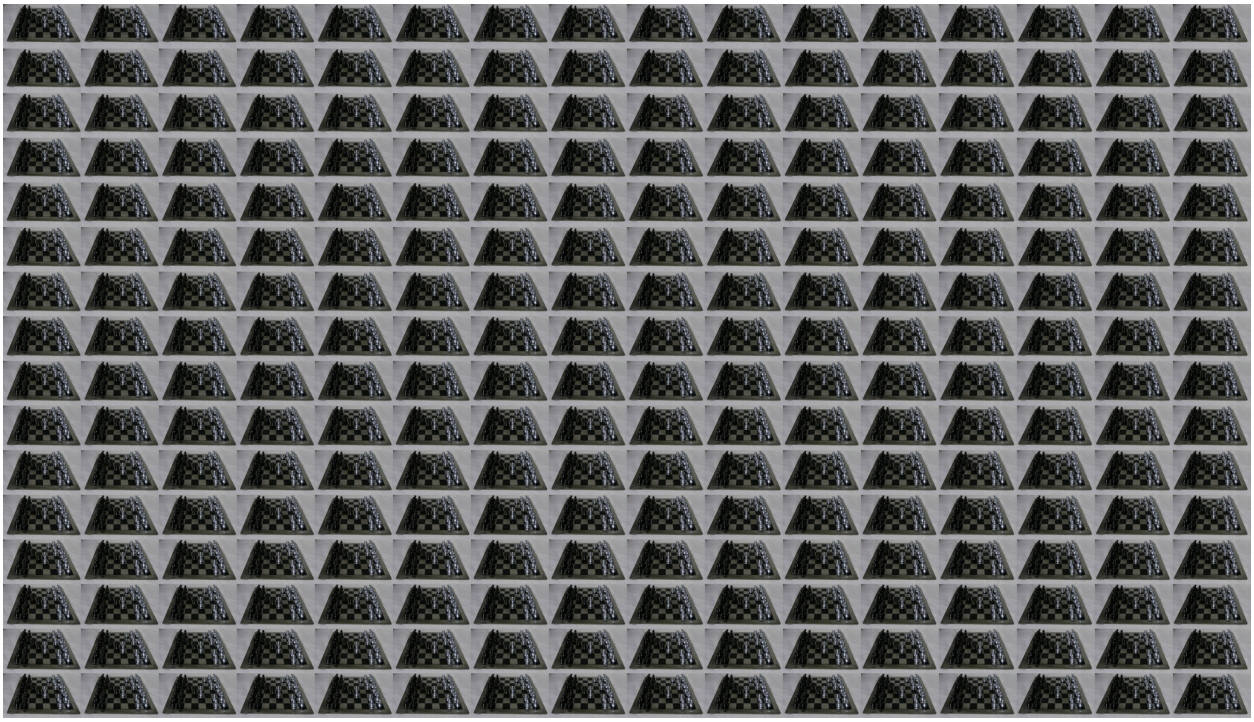


Figure 1: Mosaic of the 16×16 sub-aperture views; u increases downward and v to the right.

1.3 Refocusing and focal-stack simulation

`refocus` implements Eq. (2) as a sum over the (u, v) in the aperture A, with u and v measured from the lenslet center ($u - 7.5$, as in the hint). For each (u, v), the sub-aperture view is shifted so that refocused pixel (s, t) samples $L(u, v, s + d\,u, t - d\,v)$. The sign of the t shift is flipped from Eq. (2) because of the mirrored v axis: with it, the views of a plane at depth d line up along both axes.

- **Interpolation.** The shifts $d\,u$ and $d\,v$ are fractional, so I interpolate bilinearly. `shift_image` computes this directly as a weighted sum of the four integer shifts around $(d\,u, -d\,v)$. This gives the same values as linear `RegularGridInterpolator`, but faster.
- **Borders.** Samples that fall outside the view take the value of the nearest border pixel, so the edges of the refocused image do not darken.
- **Normalization.** The sum is divided by the number of views $|A|$, so every refocused image has the exposure of a single view, as in Fig. 3 of the assignment.
- **Aperture.** The focal stack uses the square aperture $|u|, |v| \leq 4$, that is, the central 8×8 views.
- **Depth range.** I refocus at d from -2 to 2 in steps of 0.1 (41 images). The back of the board is in focus at about $d = -0.4$, and the front pieces at about $d = 1.6$ (Fig. 2). The range extends past both ends, so every part of the scene, including the cloth behind and in front of the board, is in focus in at least one image.

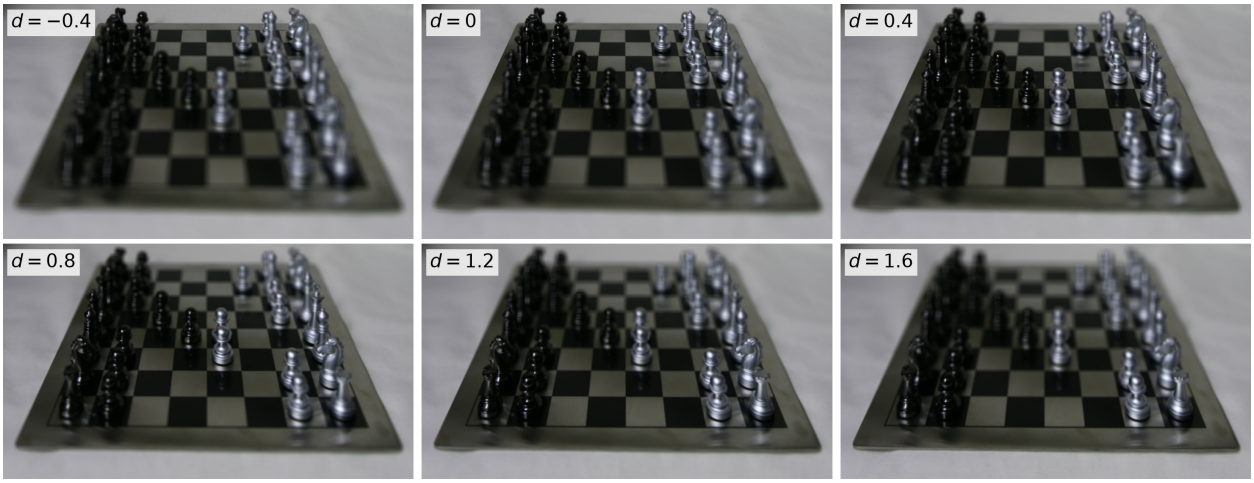


Figure 2: Refocusing with the central 8×8 views at six depths, from the back of the board ($d = -0.4$) to the front ($d = 1.6$). All 41 depths of the focal stack are in `data/part1/refocus/`.

1.4 All-in-focus image and depth from focus

`sharpness_weights` implements Eqs. (3)–(6). Each refocused image is linearized with the inverse sRGB curve and converted to XYZ luminance. The Gaussian blurs act only within each image, not across depths. Eqs. (7) and (8) are then weighted averages over the stack. I tried $\sigma_1 \in \{0.5, 1, 2, 4, 8\}$ with $\sigma_2 = 4$ (Fig. 3) and $\sigma_2 \in \{1, 2, 4, 8, 16\}$ with $\sigma_1 = 2$ (Fig. 4).

The all-in-focus image hardly depends on either value: every setting picks a sharp image at the pieces, and the crops are almost indistinguishable. The depth map depends on both:

- σ_1 sets which frequencies count as sharp, since $I_{\text{high-freq}}$ keeps the detail finer than about σ_1. With $\sigma_1 = 4$ or 8, it also keeps mid frequencies, which survive moderate defocus. The weights then peak less sharply in d, the cloth around the board becomes speckled, and the halo around the board's edge gets thicker.

- σ_2 sets the window over which the sharpness is pooled. With $\sigma_2 = 1$ or 2, the windows in flat regions (the cloth and the inside of the squares) contain too little texture, and the depth there is noise. With $\sigma_2 = 8$ or 16, the depth map is smooth, but the pieces merge into blobs and depth bleeds across their outlines.

$\sigma_1 = 2$ and $\sigma_2 = 4$ work best ($\sigma_1 = 1$ is about as good), and I use them for everything that follows.

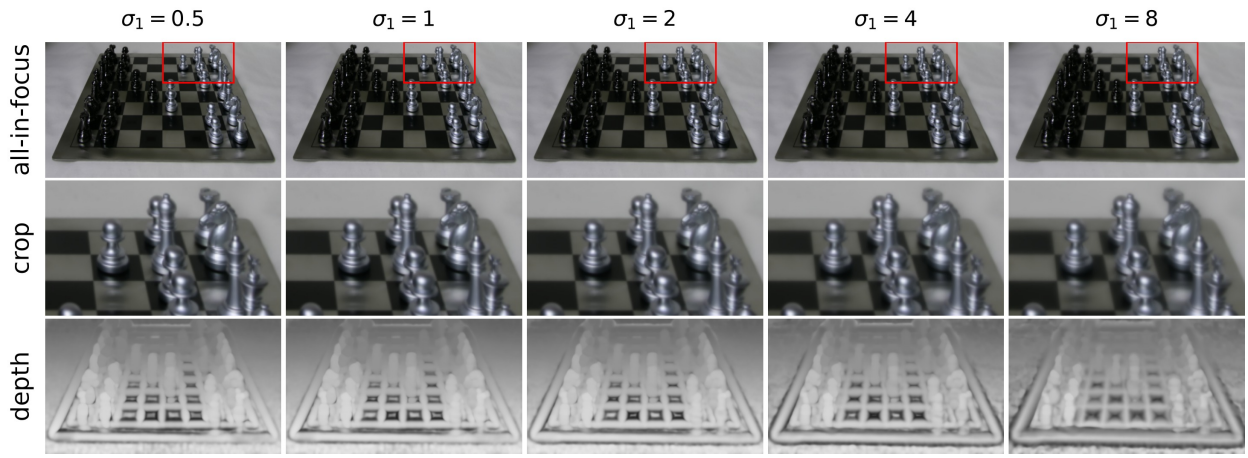


Figure 3: Varying σ_1 with $\sigma_2 = 4$: all-in-focus image (top), its red box enlarged (middle), and depth map (bottom). Depth maps show d from -2 (black) to 2 (white), so brighter is closer to the camera.

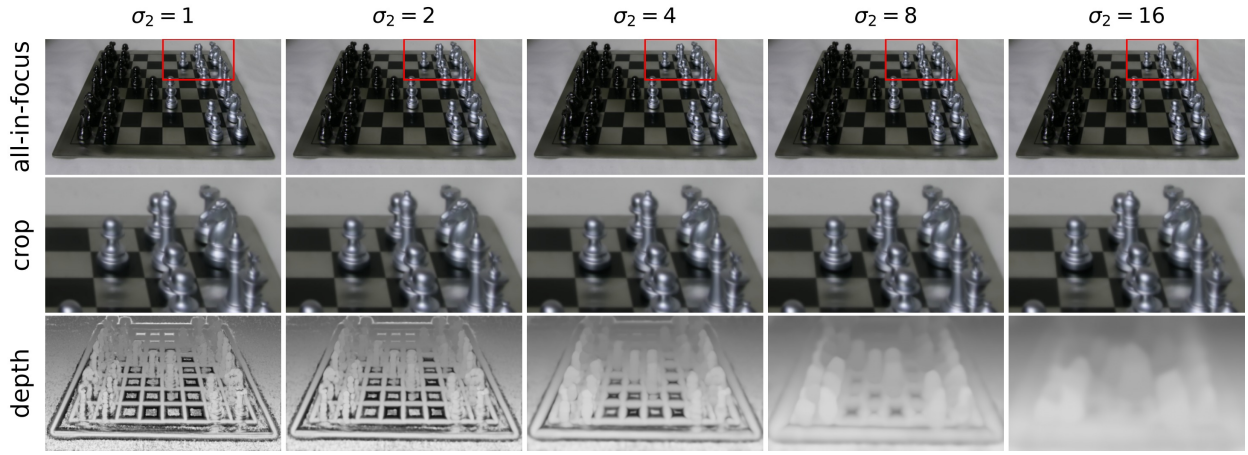


Figure 4: Varying σ_2 with $\sigma_1 = 2$, laid out as in Figure 3.

Depth errors. Depth from focus only works where the image has texture within the σ_2 window. It fails in three kinds of places:

- **Inside the checker squares.** The plain squares are dark in the depth map, which means they are estimated much farther away than the board around them. Along the middle of the board, the estimate stays near $d = 0.3$ from the middle rows to the near rows, although the board there comes into focus at up to $d \approx 1.4$. Inside a uniform square, the luminance has high frequencies only when a nearby edge (a square border or a piece) is defocused enough for its blur to reach the pixel. The weight therefore peaks at strongly defocused images, which is the wrong depth.

- **Around edges.** The board's outline appears as a thick double rim, and the pieces look wider than they are. σ_2 spreads each edge's sharpness over several pixels, and a defocused edge spreads it even further, so pixels next to an occlusion boundary take the depth of the boundary.

- **The cloth.** The white cloth around the board has almost no texture, so its depth is either noise (small σ_2) or a smooth average of its surroundings (large σ_2).

In addition, Eq. (8) is a weighted average, and the weights are never exactly zero. The estimates are therefore pulled toward the middle of the range: they span only -1.42 to 1.67.

The all-in-focus image is mostly unaffected. Where the depth is wrong because there is no texture, all the images in the stack are nearly the same, since blurring a uniform region leaves it uniform. Any weighting of them gives the right color. Depth needs to know where the sharpness peaks, while the all-in-focus image only needs the weighted images to agree. The errors show only next to edges, where the weights partly select a defocused copy of the edge. This produces the soft, slightly haloed piece outlines visible in the crops.

1.5 Focal-aperture stack and confocal stereo

I use square apertures of side $a = 2, 4, \ldots, 16$ ($|u|, |v| \le a/2$). These range from the central 2×2 views to all 16×16. Each is refocused at the same 41 depths, giving an 8×41 focal-aperture stack (stored as float32, about 1.1 GB).

`confocal_depth` implements Eqs. (7) and (8) of Hasinoff and Kutulakos:

- **Eq. (7).** Each AFI is normalized by the exitance E of each aperture. With $E = |A|$, this is exactly the division by $|A|$ that `refocus` already does, so I do not divide again. At the in-focus setting, every aperture sees the same scene point, so the normalized intensities agree across apertures (confocal constancy). At other settings, larger apertures mix in more of the neighborhood.

- **Eq. (8).** For each pixel, I score each focus setting by the variance of its AFI column across the 8 apertures, summed over the color channels. The depth is the d with the smallest variance. There is no spatial pooling: each pixel uses only its own AFI.

Figure 6 shows two AFIs. Pixel A, on the edge of a black piece at the front left, has its most consistent column at $d = 1.0$. Pixel B, on a silver piece at the back, has its most consistent column at $d = 0$. Away from those columns, the larger apertures (lower rows) blend in the background.

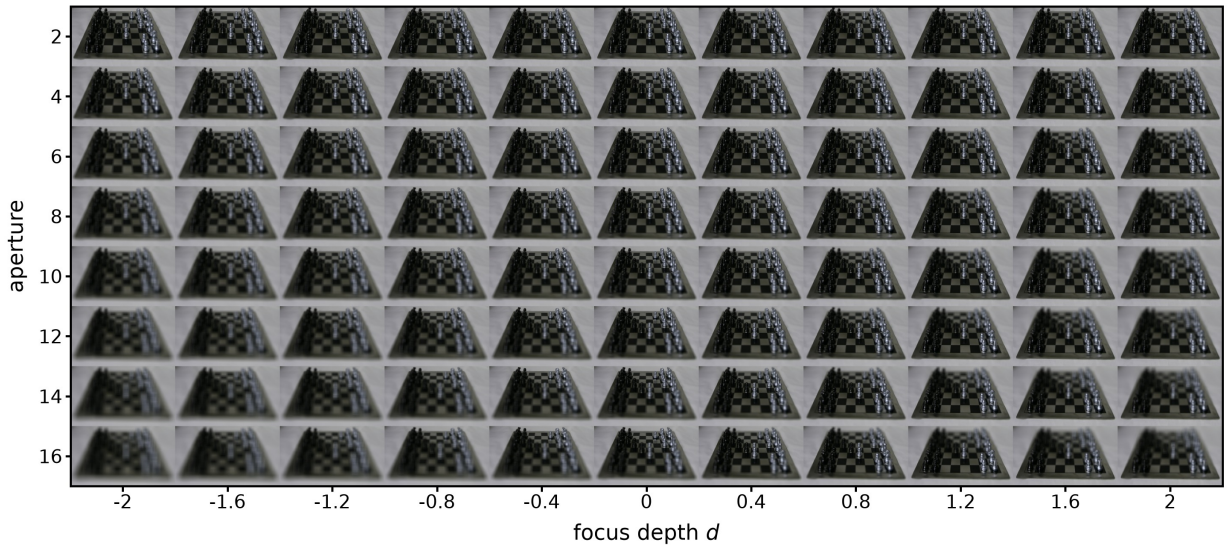


Figure 5: Collage of the focal-aperture stack: square apertures of side 2 to 16 (rows) and every fourth of the 41 focus depths (columns).

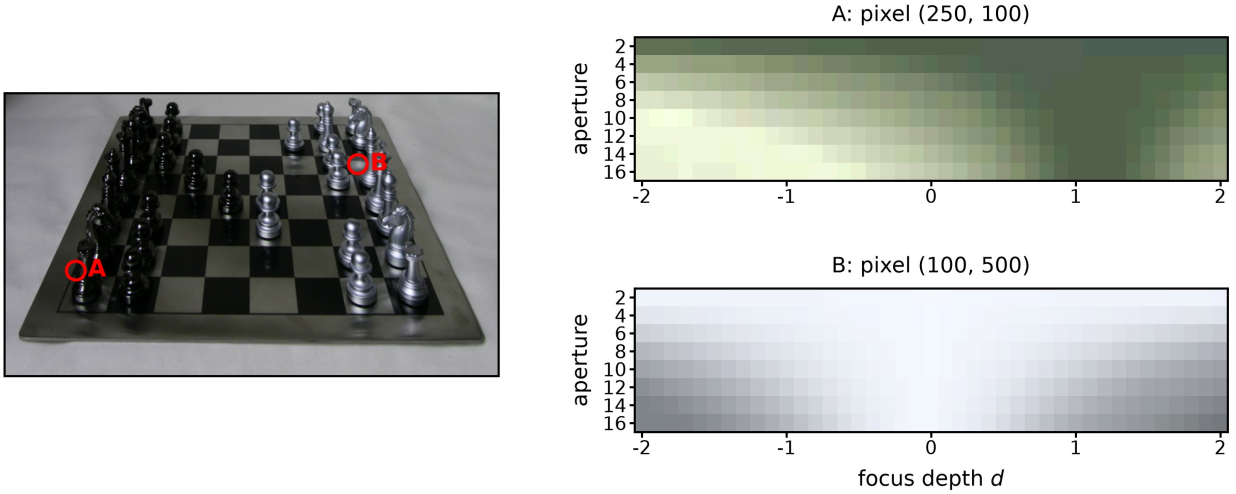


Figure 6: AFIs of the two pixels marked on the central view. Rows are apertures and columns focus depths; each AFI is scaled to its own maximum.

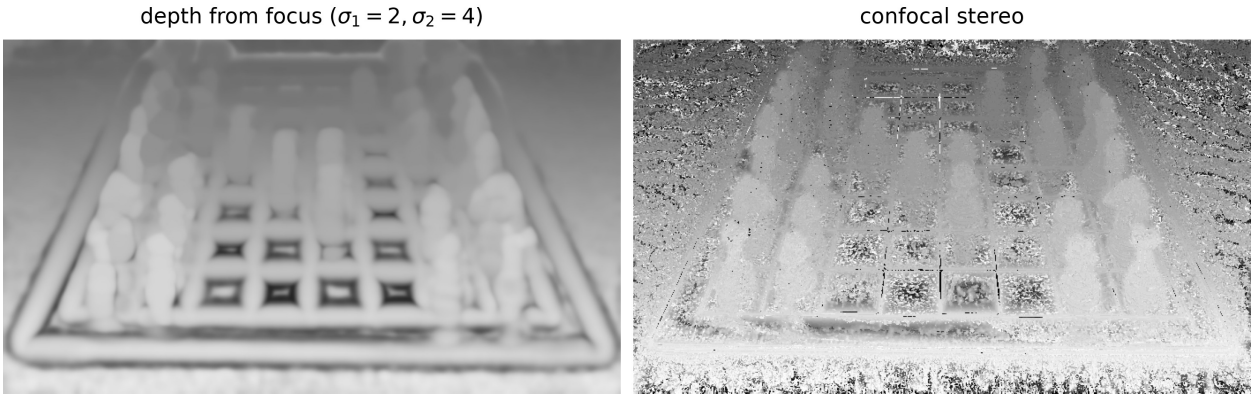


Figure 7: Depth from focus ($\sigma_1 = 2$, $\sigma_2 = 4$) and confocal stereo, on the same scale (d from -2 in black to 2 in white).

Where there is texture, the two methods agree. On the most textured quarter of the pixels (by maximum sharpness weight), their median difference is 0.09, about one step of the depth grid. At the AFI pixels, confocal stereo gives 1.0 and 0, and depth from focus gives 0.98 and -0.07. The two maps then fail in opposite ways (Fig. 7):

- **Confocal stereo** works per pixel, so it has no halos. The piece outlines are sharp, and the board's slope shows even between pieces. Along the middle of the board, its median depth rises from 0.7 to 1.4 toward the front, while depth from focus stays near 0.3. However, it is very noisy wherever the AFI is nearly the same at every focus setting, as on the cloth and inside the squares. On the least textured quarter of the pixels, the median deviation from a 5×5 median filter is 0.2 for confocal stereo and essentially zero for depth from focus, and the two maps differ by 0.44. Its depths are also restricted to the 41 settings of the grid.
- **Depth from focus** is smooth and looks cleaner, because σ_2 pools over a neighborhood. That same pooling causes its errors around edges and inside flat regions.

Overall, 27% of the pixels differ by more than 0.5. Confocal stereo is more accurate where it has signal but needs spatial aggregation to be usable. In Part 4, I blur its cost before the argmin.

2 Capture and Refocus Your Own Lightfield (3)

2.1 Capturing an unstructured lightfield

I captured three videos with my phone **[TODO: phone model, camera app, and whether focus, exposure and white balance were locked]**, each of a scene with objects at clearly different depths (Table 1). The two *chipotle* videos look out of a parking garage, past a barrier and a chain-link fence, at a garage, a row of shops and a church spire. For `chipotle.mov` I pressed the phone against the window glass and slid it in a zig-zag. **[TODO: How `chipotle_bars.mov` was moved.]** *Hamerschlag* was filmed hand-held on the CMU campus lawn, moving the phone freely in a zig-zag. The phone recorded all three as 10-bit HDR (HLG); OpenCV decodes them to 8-bit RGB, which I use directly. All three videos are in `data/part3/`. `chipotle_bars.mov` is re-encoded (HEVC, same resolution and frame rate) to fit under GitHub's 100 MB file limit.

Video	Capture	Motion	Frames used	Processed at
`chipotle.mov`	3840×2160, 30 fps, 21 s	pressed to window	every 2nd (320)	960×540
`chipotle_bars.mov`	3840×2160, 24 fps, 47 s	[TODO:]	every 3rd (375)	960×540
Hamerschlag	1920×1080, 30 fps, 29 s	hand-held	every 2nd (428)	960×540

Table 1: The captured videos. All frames are resized to 960×540 before processing.

2.2 Refocusing an unstructured lightfield

Normalized cross-correlation. Template matching runs on luminance: each frame is linearized with the inverse sRGB curve and converted to the Y channel of XYZ, as in the earlier assignments. Let the template g be $m \times m$ with $n = m^2$ pixels, and let $g_0 = g - \bar{g}$. I compute Eq. (9) from `scipy.signal.correlate2d` in `valid` mode, so that output $[i, j]$ corresponds to the template placed with its top-left corner at (i, j), plus pixel-wise operations:

- **Box filter.** $\bar{I}_t = I_t \star B$, where B is an $m \times m$ box of value $1/n$. The box is separable, so I apply it as a $1 \times m$ correlation followed by an $m \times 1$ one.
- **Numerator.** $\sum_{k,l} g_0[k,l]\,(I_t[i+k, j+l] - \bar{I}_t[i,j]) = (I_t \star g_0)[i,j] - \bar{I}_t[i,j] \sum_{k,l} g_0[k,l]$. The last sum is zero because g_0 has zero mean, so the numerator is a single correlation of the frame with g_0.
- **Denominator.** Expanding the square and using $\sum_{k,l} I_t[i+k, j+l] = n\bar{I}_t[i,j]$,

$$\sum_{k,l} (I_t[i+k, j+l] - \bar{I}_t[i,j])^2 = n\left((I_t^2 \star B)[i,j] - \bar{I}_t[i,j]^2\right),$$

  which needs one more box filter, of the squared frame. The template term $\sum_{k,l} g_0^2$ is a constant.

Then

$$h_t = \frac{I_t \star g_0}{\sqrt{\sum g_0^2 \cdot n \,\max\!\left(I_t^2 \star B - \bar{I}_t^2,\ 0\right) + \epsilon}},$$

where the max removes small negative values from rounding and $\epsilon = 10^{-9}$ avoids division by zero on flat patches. On random test images this matches OpenCV's `TM_CCOEFF_NORMED` to within 1.3×10^{-7}.

Shifting and averaging. I pick a 60×60 template in the middle frame and, in every frame, compute h_t only inside a square search window centered on the template, clipped to the frame. The argmax of h_t (Eq. (10)) is where the template's top-left corner lands in frame t, and the shift is the template's position in the middle frame minus that location. Shifts are integers, so each frame is shifted by cropping and pasting, with zeros for the pixels shifted in from outside. Instead of dividing the sum by the number of frames, I divide each pixel by the number of shifted frames that cover it, so the borders do not darken.

Search window. Following the hint, the window should be slightly larger than the template plus the largest shift. I first used a 1000-pixel window on `chipotle_bars.mov`, which is effectively the whole frame. That let 37–53 of the 375 frames per template match somewhere else entirely, which created ghosts. The true shifts in both chipotle videos are only about ±10–30 pixels, so I

use a 140-pixel window there. The phone moved much more in the Hamerschlag video (about ±90 pixels), so I use 260 pixels there.



Figure 8: `chipotle.mov`: templates (red boxes) on the middle frame, and the refocused images, from the spire (far) to the brick pillar (near). The parked car disappears from every result because it drove away during the capture.

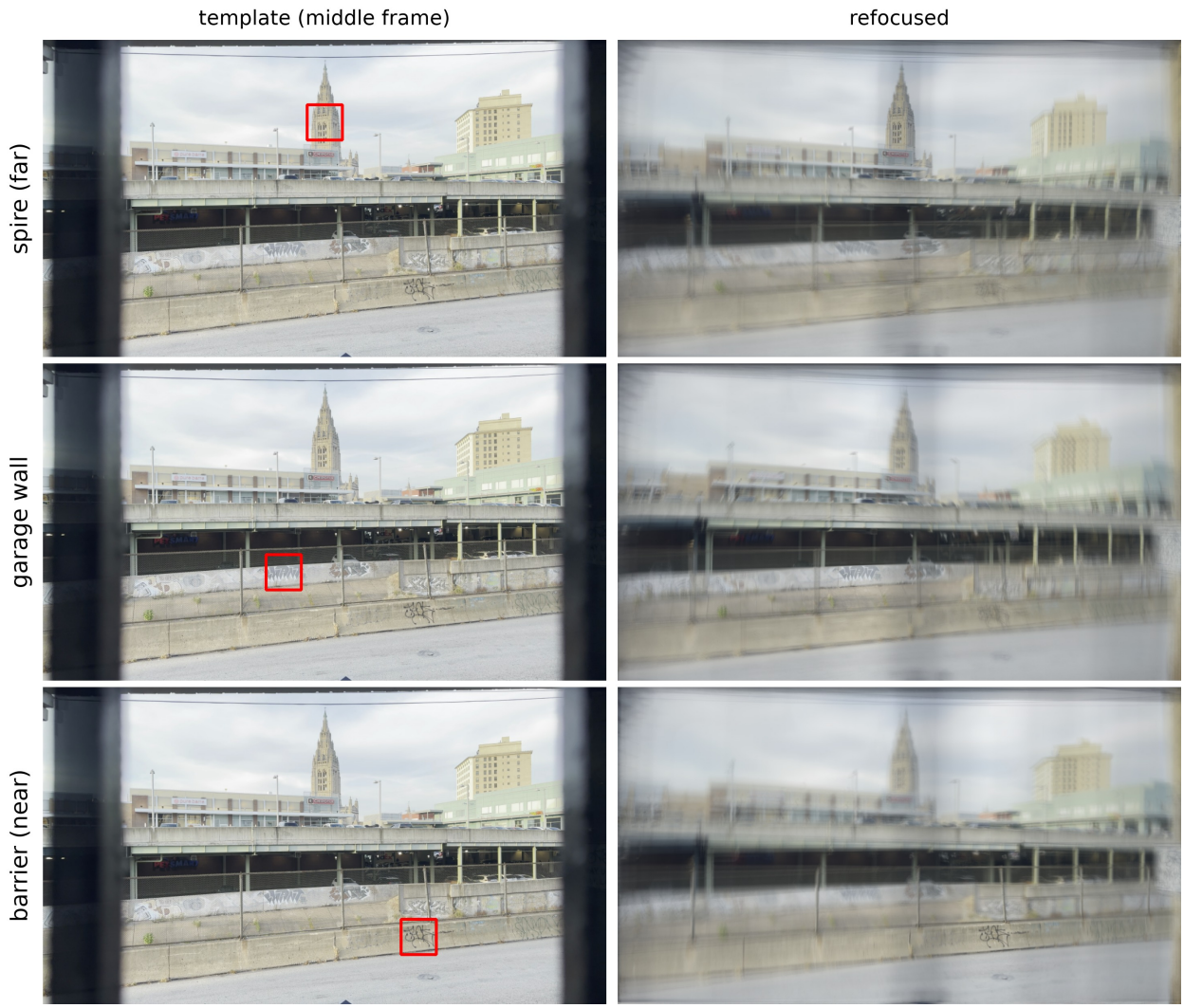


Figure 9: `chipotle_bars.mov`. The dark window bars near the camera sweep across the scene and leave a faint vertical band in the results.

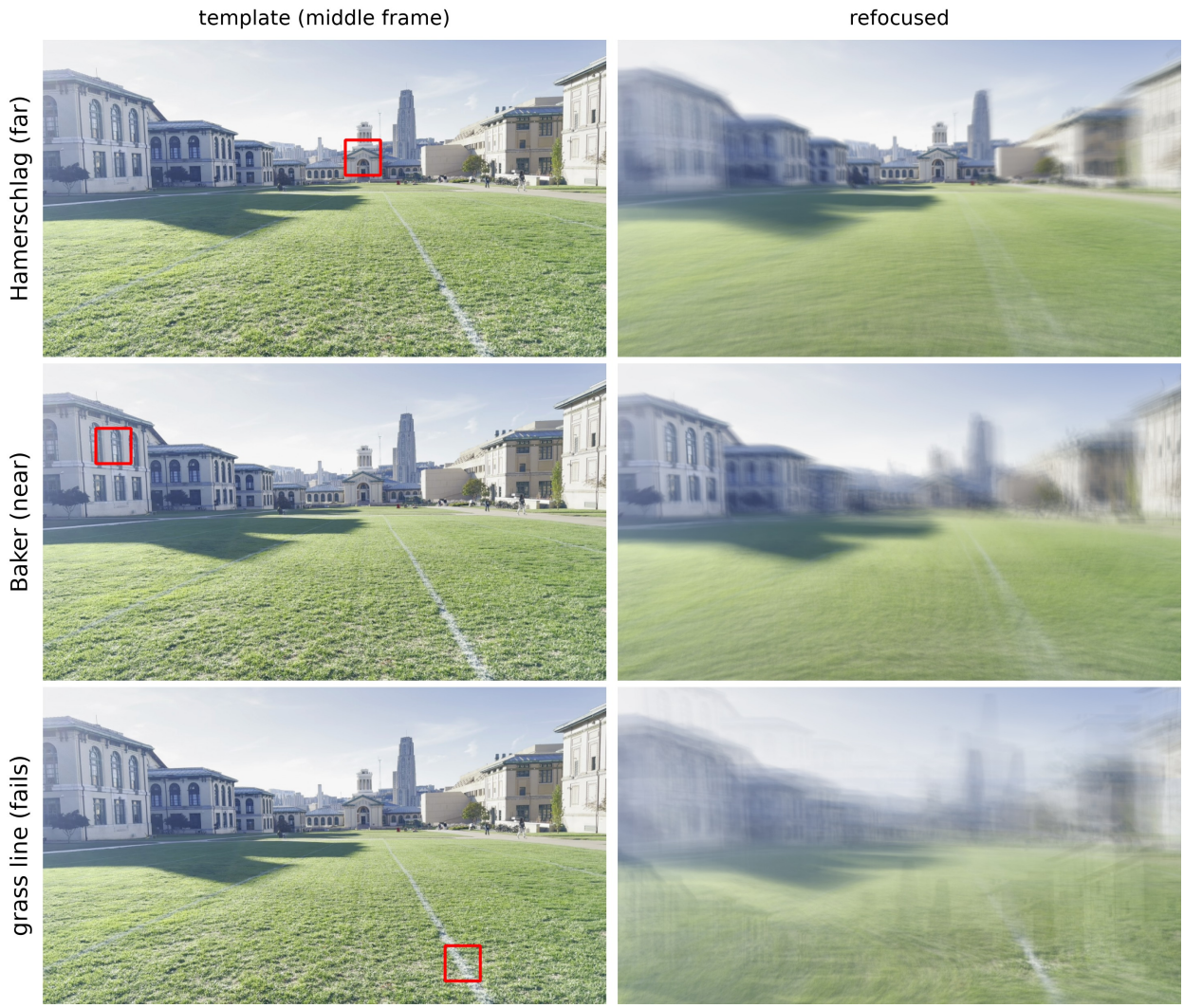


Figure 10: Hamerschlag (hand-held). Focusing on Hamerschlag Hall keeps it and the Cathedral of Learning sharp and blurs the buildings on either side. Focusing on Baker Hall (left) blurs the background. The grass-line template fails (see text).

Failures. Template matching fails when the template does not look the same in every frame:

- **Moving objects.** The parked car in `chipotle.mov` drove away during the capture, so it could not be matched (every frame hit the window edge) and it vanishes from the averages.
- **Occlusion.** In `chipotle_bars.mov` the window bars pass over the templates in about 31–35 of the 375 frames. Those frames match the wrong place, which blurs the in-focus object slightly. The barrier template in `chipotle.mov` similarly hits the window edge in 18 of 320 frames.
- **Repetitive texture and the ground plane.** The grass-line template in the Hamerschlag video loses the match in about 20% of the frames, mostly at the start of the clip, which produces the ghosted buildings in Fig. 10. Even with perfect matches, only a thin strip of the ground could be in focus, because the ground spans a continuous range of depths.

2.3 Camera rotation and the shape of the defocus

The defocus in these results does not look like a lens blur. Out-of-focus regions appear swept around the point in focus, and objects at the in-focus depth but far from the template are softer than expected (Fig. 11). Shift-and-add assumes that the camera only translates in a plane parallel to the sensor. Then every frame is a translated view, and one 2D shift aligns a whole depth plane. A hand-held phone also rotates, and the two kinds of rotation affect the result very differently:

- **Pan and tilt** (rotation about axes in the sensor plane) move the whole image by nearly the same amount, regardless of depth, for small angles. Template matching absorbs this into the per-frame shift; only the perspective change toward the image edges remains.
- **Roll** (rotation about the optical axis) rotates the image. After aligning at the template, each frame is still rotated by its roll angle θ_t about the template point, so a point at distance r from the template moves by about $r\theta_t$, perpendicular to the line to the template. Averaging frames with different θ_t smears every point along an arc around the template. A translation can only cancel this at the template itself.

To measure this, I tracked two distant points with the same NCC matching. In `chipotle.mov` these are the spire and the building 270 pixels to its right. Under pure translation they would shift identically. Instead, their relative vertical shift drifts smoothly during the clip. This corresponds to a roll with a median of 1.7° and a maximum of 3.2°, with more than 1° in 216 of 320 frames. Their relative horizontal shift implies a scale change below 0.4%, so the distance to the scene barely changed. At 3°, a point 300 pixels from the template moves by about 16 pixels. That is the same order as the real depth parallax in this video: the pillar moves about ±25 pixels relative to the spire. In the Hamerschlag video the phone moved much more: Hamerschlag Hall shifts by −78 to +53 pixels vertically and −32 to +84 horizontally (5th–95th percentile). However, the Cathedral of Learning stays within 6 pixels of Hamerschlag Hall the whole time. The large motion is therefore nearly all depth-independent pan and tilt, which template matching removes. The parallax that actually forms the refocusing is small: Baker Hall moves by only −4 to +20 and −10 to +25 pixels relative to Hamerschlag Hall.

This explains why the hand-held video refocuses about as well as the one captured against the window. Pressing the phone against the glass prevents pan, tilt and changes in distance, but it leaves the phone free to spin in the plane of the glass. Roll is exactly the rotation that a translation cannot correct. For this method, the important constraint during capture is therefore not to rotate the phone about its optical axis. Handling roll would require estimating a rotation as well as a shift for each frame, for example from two templates at the focus depth.

The camera's trajectory also acts as the synthetic aperture. Each out-of-focus point is blurred by a scaled copy of the camera path, not by a filled disk, so the blur is streaky. It is mostly horizontal, because the motion was mostly side to side.

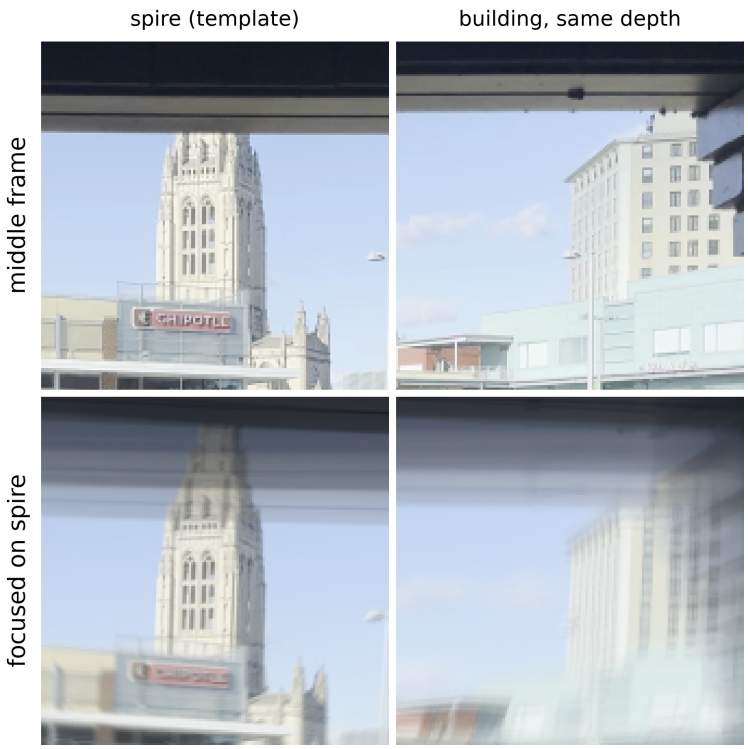


Figure 11: Crops of `chipotle.mov` focused on the spire. The spire (the template) is sharp. A building at about the same distance, 270 pixels to the right, is smeared vertically, because the camera roll rotates each aligned frame about the template.

3 Bonus: Capture and Process Your Own Focal-Aperture Stack (4)

3.1 Capture

I captured a table of pumpkins and gourds in front of a paper screen with a Nikon D3400 and a 35 mm lens, **[TODO: how the camera was mounted]**. The camera is controlled over USB with gphoto2 (`src/take_focal_stack.py`), so it is not touched during the capture. The script turns off auto ISO, fixes ISO 400, and saves every shot as both NEF and JPEG. It then sweeps the focus once for each of four apertures, f/1.8, f/4, f/9 and f/22, the widest and narrowest of the lens and two in between:

- **Exposure.** The shutter speed is scaled by N^2 from that of the first aperture, so that t/N^2 stays constant, and rounded to the closest speed the camera offers: 1/125 s, 1/25 s, 1/5 s and 1.3 s.
- **Focus.** Manual focus driving (`manualfocusdrive`) only works in live view, which only lasts for one gphoto2 session, so each sweep runs as a single `gphoto2 --shell` session. It first drives the focus to the end of its range, repeating a large step until the camera refuses, so that every aperture starts from the same focus setting. It then takes 30 shots, stepping the focus by the same amount between shots. The camera can drop out of live view after a capture, so live view is turned back on before each step; the script warns if a step still fails, since that would put that aperture's focus settings out of line with the others.

This gives 4×30 images. Focus setting 0 is focused farthest, on the screen, and the focus moves toward the camera with the index (Fig. 12). I only use settings 5 to 22, where something in the scene is in focus.

3.2 Radiometric calibration and alignment

`src/focal_aperture.py` reads the NEFs linearly with `rawpy` (half size, camera white balance, no gamma or auto-brightening), resizes them to 800 pixels wide, and divides each by its t/N^2 from the EXIF. This does not make the apertures consistent:

- **Per-shot brightness.** Defocus only spreads light around, so the mean over the central half of the frame should be the same in every shot. After dividing by t/N^2, it still jumps by up to 34% between shots of one aperture (at f/22, with 1.3 s exposures), and the four apertures' medians differ by up to 30%. I scale every shot to the same central mean.
- **Vignetting.** The wide apertures vignette more, which changes the exitance per pixel. For each aperture, I average its images over all focus settings, blur the average heavily ($\sigma = 40$ pixels) so defocus does not matter, and divide the aperture's images by the ratio of this to the same average for f/22.
- **Magnification.** Refocusing changes the magnification of the lens. I estimate it with ECC (`cv2.findTransformECC`) on the f/22 sweep, which is sharp at every focus setting, aligning each focus setting to the middle one and starting each estimate from its neighbor's. I reduce each affine warp to the closest scale plus translation, since refocusing should not rotate or shear the image. The scale grows steadily from 0.995 to 1.019 over the 30 settings, about 10 pixels at the edges of the 800-pixel image, and the shifts stay within a few pixels. Every aperture's image at a focus setting is warped with that setting's estimate, and the border that the warps leave undefined is cropped, giving 782×516 images.



Figure 12: Example frames at the widest and narrowest apertures, after calibration and alignment. Focus moves from the screen (setting 5) to the front of the table (setting 22).

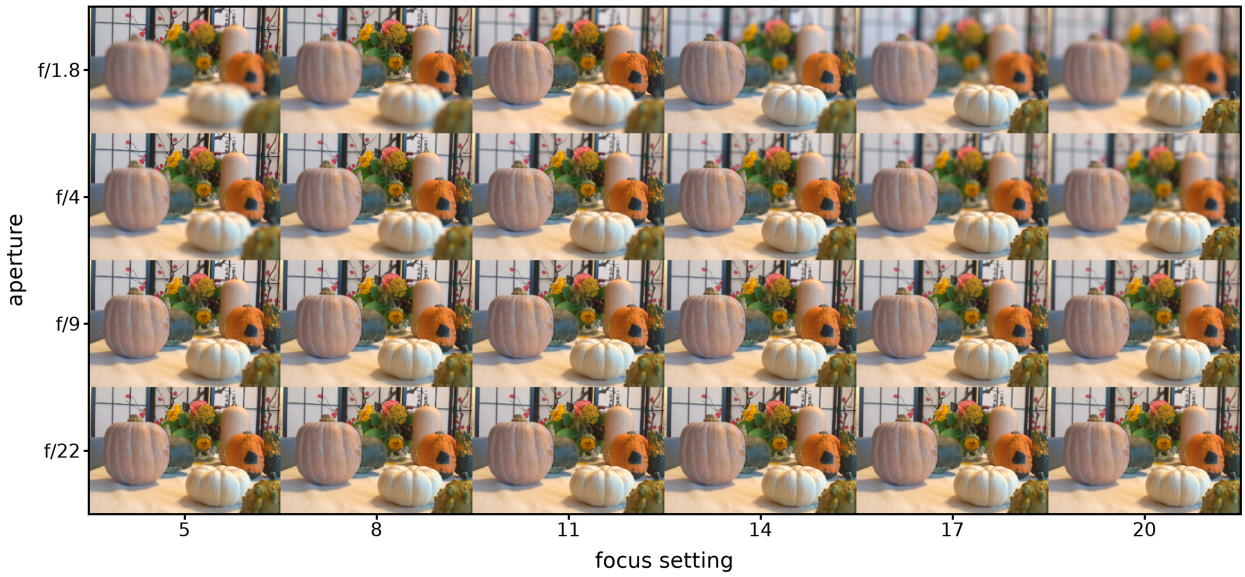


Figure 13: Collage of the captured focal-aperture stack: every third focus setting from 5 to 22.

3.3 All-in-focus image and depth maps

Depth from focus and the all-in-focus image use the same code as Part 1 (Eqs. (3)–(8), $\sigma_1 = 2$, $\sigma_2 = 4$) on the f/1.8 sweep. Confocal stereo uses the same Eqs. (7) and (8) of Hasinoff and Kutulakos as Part 1, on all four apertures, with the per-pixel cost blurred by $\sigma = 2$ before the argmin. Depth is in units of focus setting.



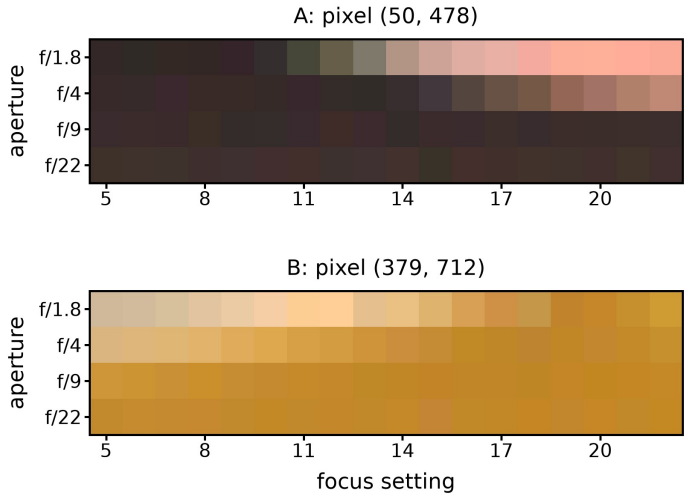


Figure 14: AFIs of a branch on the screen (A) and the bumpy gourd at the front (B). Each AFI is scaled to its own maximum.

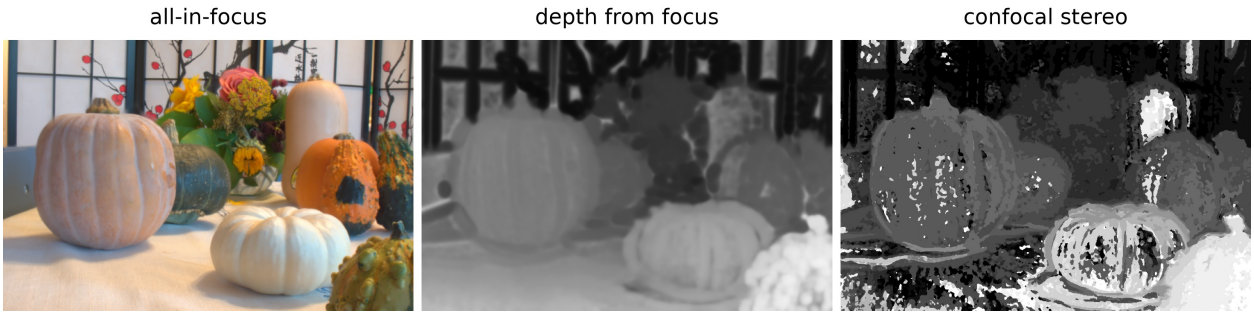


Figure 15: All-in-focus image, and depth from focus and confocal stereo depth from focus setting 5 (black) to 22 (white), so brighter is closer.

The two maps agree on the overall layout. The screen is farthest (about setting 6 in both), the gourd at the front right is nearest, and the large pumpkin is in between. They differ in the same ways as in Part 1, and the differences are larger here: the median difference is 1.6 settings, and 22% of the pixels differ by more than 4.

- **Depth from focus** is smooth, with soft boundaries and halos around the pumpkins. It also compresses the range, since the weighted average spans only 5.6 to 21.2. The front gourd, which is sharpest at setting 22 (Fig. 12), gets 18.6. Its depths are right on the white pumpkin and the large pumpkin, which are both sharp at setting 14 (estimates 15.1 and 13.3).
- **Confocal stereo** has crisp outlines and reaches the ends of the range (21 on the front gourd). However, it takes only the 18 values of the focus settings, and 13% of its pixels land on the extremes 5 or 22. It also makes large errors on smooth, bright surfaces. The white pumpkin gets 19 instead of about 14, and the top of the butternut squash, which is behind the other gourds, gets 20 (depth from focus: 12). It is also speckled on the screen and the large pumpkin.

Confocal stereo is less reliable here than on the synthetic stack, for three reasons:

- It compares raw intensities across apertures, so any residual calibration error (per-shot brightness, vignetting, alignment) adds variance at every focus setting. Depth from focus uses one aperture and only compares sharpness.
- Only four apertures are available, compared with eight in Part 1.
- The depth of field at f/9 and f/22 is so deep that those images barely change over the sweep (the two lower rows of both AFIs in Fig. 14 are nearly constant). Most of the information therefore comes from f/1.8 and f/4.